

Experiment No.:7

Simulation of a Smart City Traffic Management System Using Python (Google Colab)

Aim

To develop and simulate a Smart City Traffic Management System using Python in Google Colab that dynamically controls traffic lights based on vehicle density, reduces traffic congestion, and displays real-time traffic conditions through graphical visualization.

Objective

The objective of this experiment is to simulate a Smart City Traffic Management System using Python in Google Colab. The system continuously monitors the number of vehicles waiting in four traffic lanes (North, South, East, and West). During each traffic cycle, new vehicles arrive randomly, and the system identifies the lane with the highest traffic density. The traffic signal automatically turns green for the busiest lane, allowing vehicles to pass while the remaining lanes remain red. The updated traffic conditions are displayed using live bar graphs. This experiment demonstrates dynamic traffic signal control, traffic density analysis, congestion management, and real-time visualization in a simulated smart city environment.

Software Requirements

- Google Colab
- Python 3.x

Python Libraries Used

- matplotlib
- random
- IPython.display
- time

```
# =====  
  
# Smart City Traffic Management System  
  
# Dynamic Density-Based Traffic Light Controller  
  
# Google Colab Version  
  
# =====  
  
import random  
  
import matplotlib.pyplot as plt  
  
from IPython.display import clear_output  
  
import time  
  
  
# Lane names  
  
lanes = ["North", "South", "East", "West"]  
  
  
# Initial queue of cars  
  
queues = [5, 2, 8, 3]  
  
  
# Number of traffic cycles  
  
simulation_steps = 20  
  
  
for step in range(1, simulation_steps + 1):  
  
    # Random new cars arrive  
  
    new_cars = [random.randint(0, 5) for _ in range(4)]  
  
    queues = [queues[i] + new_cars[i] for i in range(4)]
```

```
# Find busiest lane

active_lane = queues.index(max(queues))

active_lane_name = lanes[active_lane]


# Cars cleared

cars_cleared = min(queues[active_lane], random.randint(4, 8))

queues[active_lane] -= cars_cleared


# Clear previous graph

clear_output(wait=True)


# Colors

colors = ['red', 'red', 'red', 'red']

colors[active_lane] = 'green'


# Plot graph

plt.figure(figsize=(8,5))

plt.bar(lanes, queues, color=colors)


plt.ylim(0,30)

plt.ylabel("Number of Cars Waiting")

plt.xlabel("Traffic Lanes")

plt.title(f"Traffic Cycle {step}: Green Light -> {active_lane_name}")


# Display values on bars

for i, value in enumerate(queues):
```

```
plt.text(i, value+0.5, str(value),
        ha='center', fontsize=12)

plt.figtext(
    0.5,
    0.01,
    f"Action: Cleared {cars_cleared} cars from {active_lane_name}",
    ha='center',
    fontsize=12,
    bbox=dict(facecolor='white', edgecolor='black')
)

plt.show()

# Wait before next cycle
time.sleep(1.5)

print("Simulation Completed Successfully!")
```

Theory

A Smart City Traffic Management System improves road traffic by using intelligent traffic signal control based on vehicle density. Instead of assigning fixed signal timings, the system continuously monitors the number of vehicles waiting in each lane and dynamically gives priority to the lane with the highest traffic density.

In this simulation, four traffic lanes (North, South, East, and West) are considered. Random numbers of vehicles arrive at each lane during every traffic cycle. The lane with the highest

queue is selected, and its traffic signal turns green to clear vehicles. The remaining lanes continue to wait. The updated traffic conditions are displayed through live bar graphs, allowing easy visualization of traffic flow.

This simulation demonstrates the working principle of an intelligent traffic management system without requiring physical traffic sensors.

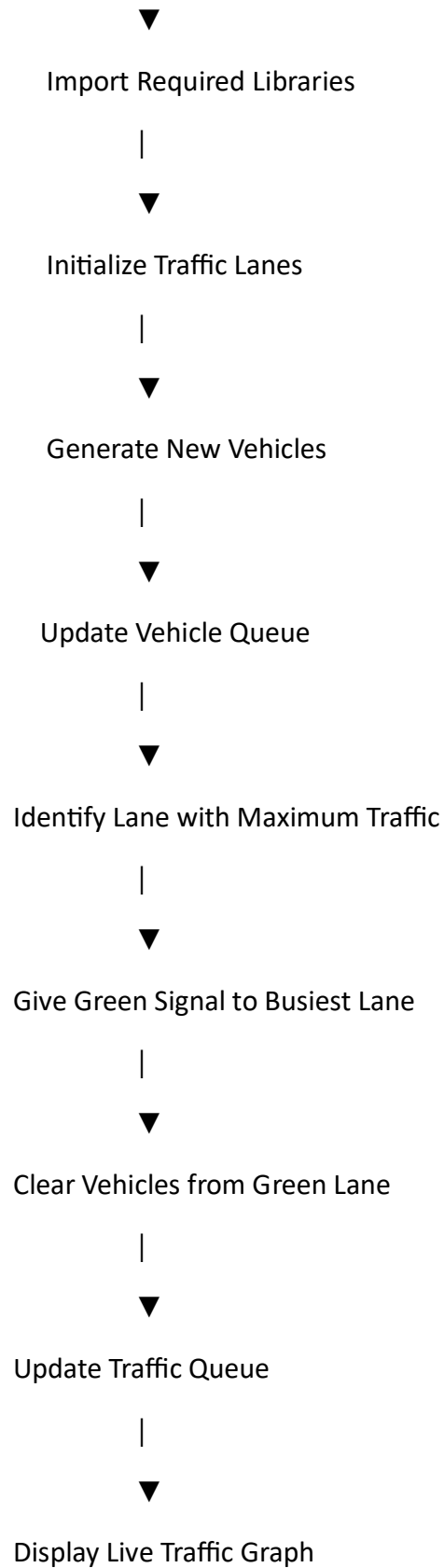
Algorithm

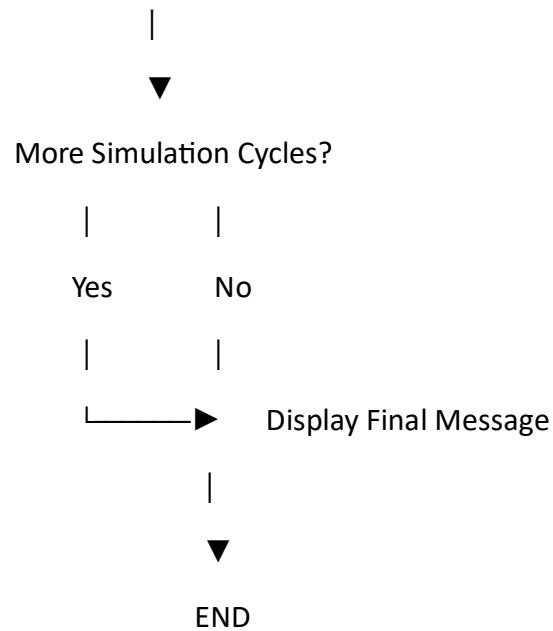
1. Start the program.
2. Import the required Python libraries.
3. Define four traffic lanes.
4. Initialize the number of waiting vehicles.
5. Set the number of simulation cycles.
6. Generate random new vehicles for each lane.
7. Update the vehicle queues.
8. Identify the lane with the highest traffic density.
9. Turn the traffic signal green for the busiest lane.
10. Clear a random number of vehicles from the selected lane.
11. Update the traffic queue.
12. Display the live traffic graph.
13. Wait for the next traffic cycle.
14. Repeat until all simulation cycles are completed.
15. Display "Simulation Completed Successfully".
16. Stop the program.

Flowchart

START

|





Traffic Parameters Used

Parameter	Description
Traffic Lanes	North, South, East, West
Initial Queue	5, 2, 8, 3 Vehicles
Simulation Steps	20 Cycles
New Vehicle Arrival	0–5 Vehicles
Vehicles Cleared	4–8 Vehicles

Program Description

The Python program simulates a Smart City Traffic Management System by monitoring four traffic lanes. During each simulation cycle, random numbers of vehicles arrive at every lane. The system compares the traffic density in all lanes and identifies the busiest lane. The traffic light automatically turns green for that lane, allowing several vehicles to pass while the remaining lanes continue waiting. The vehicle queues are updated after each cycle, and a live bar graph displays the current traffic condition. The process repeats for 20 traffic cycles, demonstrating intelligent traffic signal control based on real-time traffic density.

Input

- Traffic Lanes = North, South, East, West
- Initial Vehicle Queue = 5, 2, 8, 3
- Simulation Steps = 20
- Random Vehicle Arrival = 0–5
- Vehicles Cleared = 4–8

Output

- Live Traffic Density Bar Graph
- Green Signal for Highest Density Lane
- Updated Vehicle Queue
- Number of Vehicles Cleared
- Simulation Completion Message

Sample Output

Cycle Green Signal Vehicles Cleared Queue After Update

1	East	6	North=7, South=4, East=5, West=5
2	North	5	North=4, South=6, East=8, West=7
3	East	7	North=6, South=8, East=3, West=8
4	West	5	North=8, South=10, East=5, West=6

Observations

- Vehicle queues are updated during every traffic cycle.
- New vehicles arrive randomly.

- The busiest lane always receives the green signal.
- Vehicle congestion decreases after clearing traffic.
- Live graphs display traffic density effectively.
- The simulation demonstrates dynamic traffic signal control.

Applications

- Smart City Traffic Management
- Intelligent Transportation Systems (ITS)
- Smart Traffic Signal Control
- Urban Traffic Monitoring
- Road Congestion Management
- IoT-Based Smart Cities
- Traffic Flow Analysis
- Transportation Research

Advantages

- Reduces traffic congestion.
- Dynamic traffic signal control.
- Improves traffic flow.
- Easy to understand and implement.
- Provides real-time graphical visualization.
- Cost-effective educational simulation.
- Can be extended with IoT traffic sensors.
- Suitable for smart city research.

Limitations

- Uses simulated traffic data.
- No real traffic sensors are connected.
- Vehicle arrivals are randomly generated.
- Does not consider emergency vehicles.
- Intended only for educational purposes.

Result

The **Smart City Traffic Management System** was successfully simulated using Python in Google Colab. The system dynamically monitored vehicle density across four traffic lanes, identified the busiest lane during each traffic cycle, and allocated the green signal accordingly. Vehicle queues were updated after clearing traffic, and the changing traffic conditions were visualized using live bar graphs. Thus, the experiment successfully demonstrated dynamic traffic signal control, traffic density analysis, congestion management, and real-time visualization in a simulated smart city environment.

